

DBMS Indexing Numerical – Part 2

Dense vs. Sparse Indexing & Reduced Disk I/O Cost Analysis

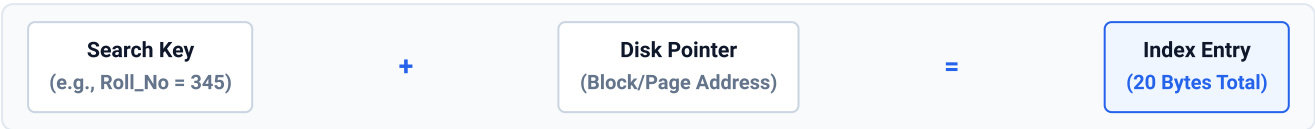
1. GIVEN PARAMETERS BASELINE

Building directly upon Part 1 calculations, we evaluate the system performance when adding an explicit index file on secondary storage.

BLOCK SIZE (B) 1,000 Bytes	RECORD SIZE (R) 250 Bytes	TOTAL DATA BLOCKS 2,500 Blocks	INDEX ENTRY SIZE 20 Bytes
-------------------------------	------------------------------	-----------------------------------	------------------------------

2. INDEX ARCHITECTURE & ENTRIES PER BLOCK

An index entry is a light key-pointer pair that maps search keys to physical data block addresses:



Index Entries Per Block = Index Block Size / Index Entry Size = 1000 / 20 = 50 entries / index block

3. DENSE VS. SPARSE INDEXING MECHANICS

A. Dense Index

Contains **one index entry for every single data record** in the table.

Total Entries = 10,000
Dense Index Blocks = 10,000 / 50 = 200 index blocks

B. Sparse Index

Contains **one index entry per data block** (requires data to be sorted/ordered by search key).

Total Entries = 2,500 (equal to total data blocks)
Sparse Index Blocks = 2,500 / 50 = 50 index blocks

4. DENSE VS. SPARSE FEATURE COMPARISON

Feature / Parameter	Dense Index	Sparse Index
Total Index Entries	10,000 entries (1 per record)	2,500 entries (1 per block)
Entries / Index Block	50	50
Index Blocks Required	200 blocks	50 blocks
Requires Ordered Data?	Not strictly required	Yes (Mandatory)
Overhead / Storage Size	Larger storage footprint	Smaller storage footprint

5. SEARCH COST CALCULATION (INDEX ACCESS + DATA FETCH)

Search involves binary search over the ordered index blocks to find the target key pointer, plus **+1 final data block access** to retrieve the record.

A. Searching with Sparse Index

Index Blocks = 50
Index Lookup Steps = $\lceil \log_2(50) \rceil = 6$ block accesses (since $2^5 = 32 < 50 \leq 64 = 2^6$)
Total Sparse Cost = 6 (Index Accesses) + 1 (Data Block Access) = **7 block accesses**

B. Searching with Dense Index

Index Blocks = 200
Index Lookup Steps = $\lceil \log_2(200) \rceil = 8$ block accesses (since $2^7 = 128 < 200 \leq 256 = 2^8$)
Total Dense Cost = 8 (Index Accesses) + 1 (Data Block Access) = **9 block accesses**

6. GRAND COMPARISON ACROSS METHODS

Search Approach	Index Blocks	Index Binary Search Steps	Data Block Access	Total I/O Cost
No Index (Ordered Binary Search)	N/A	$\lceil \log_2(2500) \rceil \approx 12$	Included	≈ 12 accesses
Dense Indexing	200	8	+1	≈ 9 accesses
Sparse Indexing	50	6	+1	≈ 7 accesses

Why Indexing Drastically Reduces I/O Cost

Index blocks are physically much smaller and more compact than data blocks (20B entry vs 250B record). This allows 50 index entries to fit in one block compared to only 4 data records per block, drastically shrinking the binary search space tree size!

7. ESSENTIAL FORMULA CHEAT SHEET

Core Formulas to Remember

- Records per Data Block** = $\lfloor \text{Block Size} / \text{Record Size} \rfloor$
- Entries per Index Block** = $\lfloor \text{Index Block Size} / \text{Index Entry Size} \rfloor$
- Dense Index Blocks** = Total Records / Entries per Index Block
- Sparse Index Blocks** = Total Data Blocks / Entries per Index Block
- Total Search Cost Formula** = $\lceil \log_2(\text{Index Blocks}) \rceil + 1$

8. QUICK EXAM RULES & TRAPS

- Golden Rule:** Dense maps 1 entry per *record*; Sparse maps 1 entry per *data block*.
- Don't forget the +1:** The +1 term in search cost accounts for fetching the actual record block from disk after locating its pointer.
- Sparse Prerequisite:** Sparse indexes always require the underlying data file to be sorted/ordered by search key.

